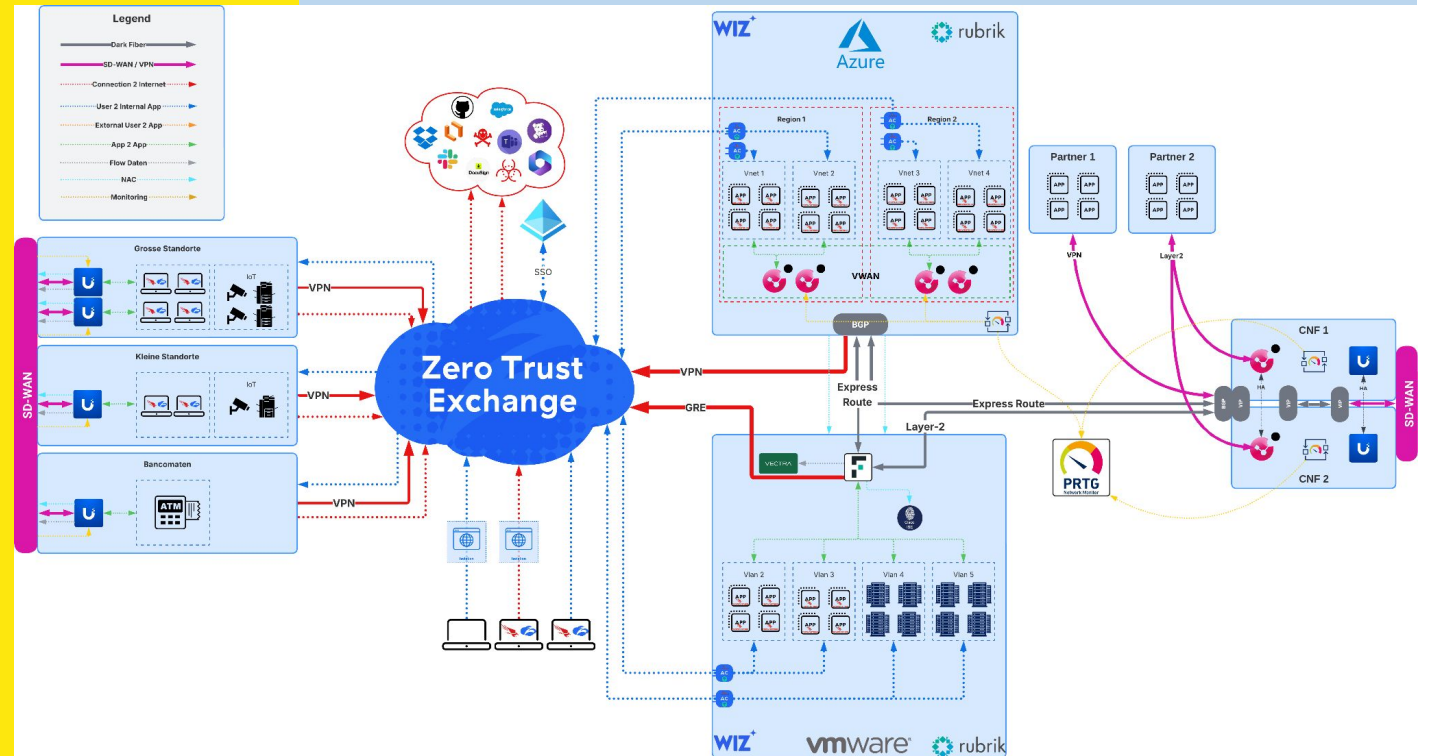


Public Clouds

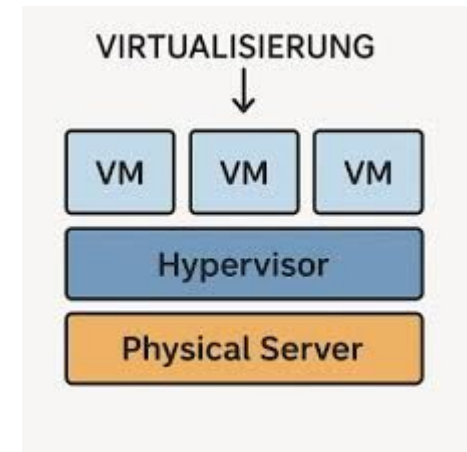


Cloud Computing: Definition & Eigenschaften

- Cloud Computing ist die Nutzung von IT-Ressourcen (z. B. Rechenleistung, Speicher, etc.) aus Rechenzentren eines Anbieters, die je nach Bedarf bereitgestellt und nach tatsächlicher Nutzung bezahlt werden (mieten statt kaufen).
- Typische Eigenschaften:
 - **Selbstbedienung:** Nutzer können Server/Speicher selbst starten und stoppen, ohne Ticket an die IT.
 - **Anbindung meist über das Internet** (modern mit SSE oder SASE, alt mit VPNs über Perimeter)
 - **Gemeinsam genutzte Ressourcen:** Viele Kunden teilen sich eine grosse Ressourcengrundlage im Rechenzentrum(Multi-Tenancy).
 - **Schnell skalierbar:** Ressourcen können automatisch schnell hoch- und runtergefahren werden (z. B. mehr Server bei Lastspitzen).
 - **Gemessene Nutzung:** Nutzung wird genau gemessen (CPU-Stunden, GB Speicher, Netzwerktraffic) und darauf basiert die Abrechnung.

Virtualisierung: Grundlagen

- Ziel: physische Ressourcen in logische Einheiten abstrahieren
- **Bereiche:**
 - Compute-Virtualisierung (VMs, Container, Serverless)
 - Netzwerk-Virtualisierung (SDN, Routings, Security-Groups)
 - Storage-Virtualisierung (Volumes, Object Storage)
- **Eigenschaften:**
 - dichte Konsolidierung von Workloads
 - schnelle Bereitstellung via API / Automatisierung (IaC)
 - Multi-Tenancy und Isolation als Grundprinzip in Clouds
- **Basis für:** IaaS, PaaS, SaaS, moderne Cloud-Netzwerke

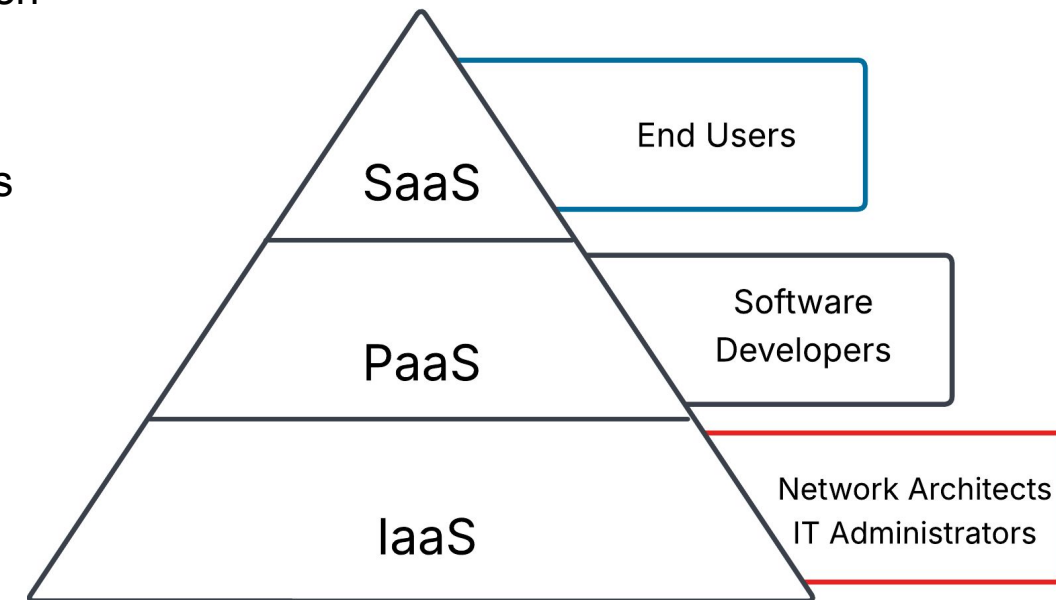


Cloud Bereitstellungsmodelle

Modell	Beschreibung	Kontrolle	Einsatzszenarien
Public Cloud	Nutzung vollständig vom Provider betriebener Ressourcen in einer Multi-Tenant-Umgebung; Zugriff über öffentliche Netzwerke; elastische Skalierung.	Niedrig bis mittel (Konfiguration ja, physische Kontrolle nein)	Breite Workload-Palette, variable Lasten, schnelle Bereitstellung, Innovation
Private Cloud	Cloud-Architektur auf dedizierter Hardware für eine einzelne Organisation (On-Prem oder Hosted); kontrollierter Zugriff und volle Governance.	Sehr hoch	Hohe Compliance-/Sicherheitsanforderungen, Legacy-Integration, interne Rechenzentren
Hybrid Cloud	Kombination aus Private und Public Cloud mit standardisierter Integration (Netzwerk, IAM, Policies, Workload-Portabilität).	Mittel	Unternehmen mit regulatorischen Vorgaben, graduelle Cloud-Migration, Datenlokalität
Community Cloud	Gemeinsame Infrastruktur für mehrere Organisationen mit gleichen Anforderungen (z. B. MSP Provider).	Gering	Kleinstunternehmen ohne IT die eine "Managed Cloud" beziehen

Cloud Service-Modelle: IaaS, PaaS, SaaS

- **Ziel:** IT-Ressourcen in verschiedenen Abstraktionsebenen aus der Cloud beziehen
- **IaaS** – Infrastructure as a Service
 - Provider stellt: Rechenzentrum, physische Hardware, Virtualisierung, Basis-Netzwerk
 - Kunde verwaltet: OS, Patches, Middleware, Applikationen, Daten
 - Beispiele: AWS EC2, Azure VMs, GCP Compute Engine
- **PaaS** – Platform as a Service
 - Provider stellt zusätzlich: OS, Netzwerk, Datenbanken, Patches
 - Kunde verwaltet: nur Applikationen & Daten
 - Beispiele: Wordpress Instanz, Nextcloud, etc.
- **SaaS** – Software as a Service
 - Provider betreibt komplette Applikation
 - Du verwaltest: Benutzer/Zugriffe, Dateninhalte
 - Beispiele: Microsoft 365, Salesforce, Google Workspace
- **Merke:** Von IaaS → PaaS → SaaS sinkt der Betriebsaufwand, steigt aber die Abhängigkeit vom Provider.



Compute-Modelle: VMs, Container, Serverless

- **Virtuelle Maschine (VM)**
 - Vollständiges Gast-OS, virtuelle Hardware (vCPU, vRAM, vNIC, vDisk)
 - Eher „schwergewichtig“, Startzeit Sekunden–Minuten
 - Hoher operativer Aufwand (Integration, Härtung, Patching, Skalierung, Config-MGMT, etc.)
- **Container**
 - Teilen sich Kernel des Host-OS, isolieren nur User-Space
 - Gibt es abstrahiert in public clouds
 - Startzeit in Sekunden, einfache Skalierung, gut für Microservices
- **Serverless / FaaS**
 - Direkte Ausführung von Code, kein Servermanagement
 - Äusserst effizient und skalierbar, keinerlei Infrastruktur nötig, aber Security!
- Alle drei nutzen darunter Virtualisierung + SDN + Overlay-Netze im Cloud-Datacenter

VMs in der Public Cloud

- VM = klassische Compute-Einheit in IaaS
- Netzwerkanbindung:
 - mindestens eine virtuelle Netzwerkkarte (AWS ENI, Azure NIC, GCP NIC)
 - IP-Adressen aus Subnet (privat, optional public)
 - Bindung an Security Groups / NSGs / Firewall-Regeln
 - Meist klassisch im Hub-and-Spoke Modell
- Typische Konfiguration:
 - Private IP (RFC1918) für internen Traffic
 - Optional Public IP / Elastic IP für direkten Internet-Zugriff
 - OS-Firewall (iptables/Windows Firewall) als zusätzliche Schicht

Container & Kubernetes im Cloud-Netz

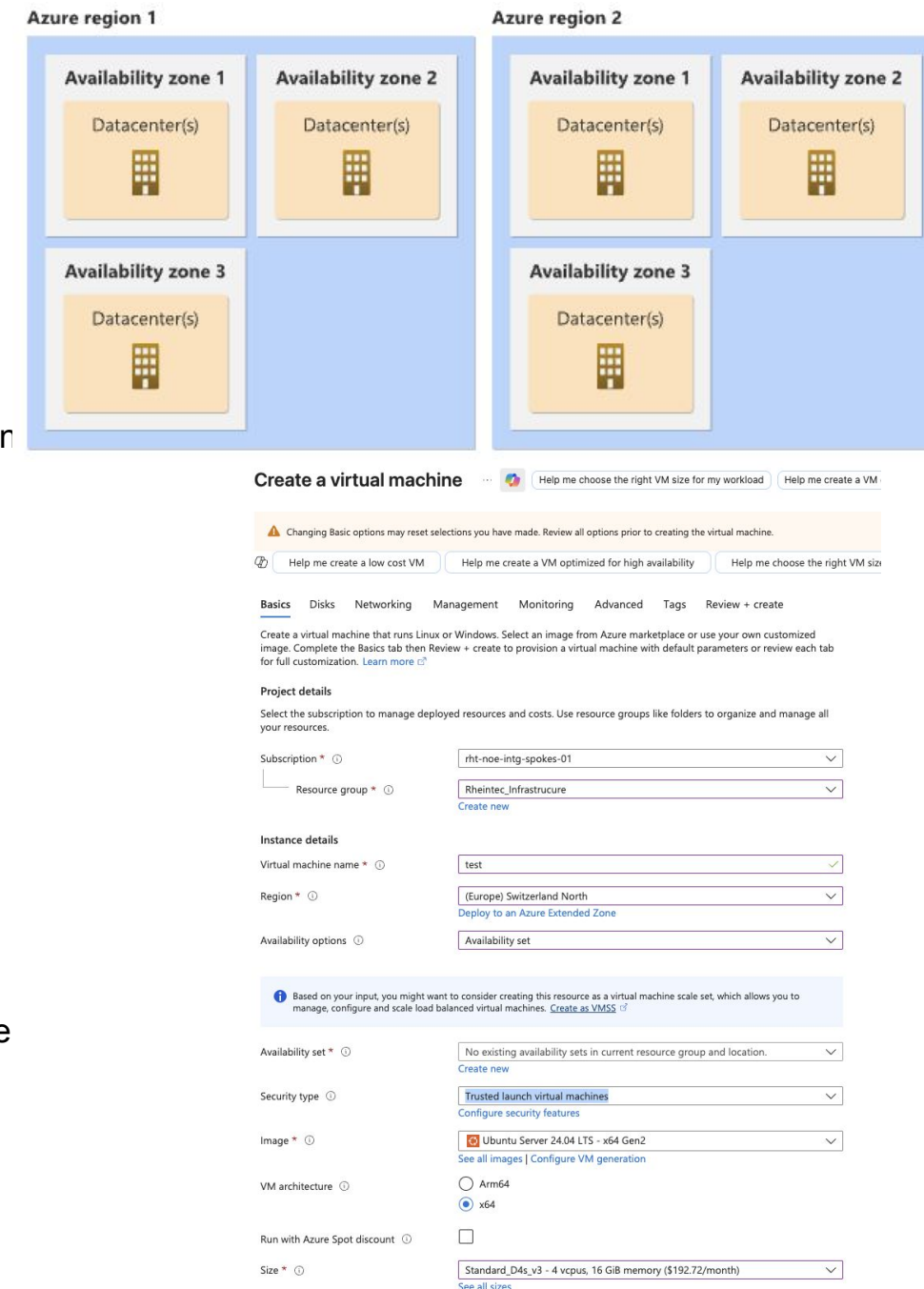
- Grundidee von Containern
 - Anwendungen werden mitsamt ihren Abhängigkeiten in Container verpackt.
 - Container sind leichtgewichtig, starten schnell und lassen sich gut automatisieren.
 - Ideal für Microservices und skalierbare Webanwendungen.
- Wie Cloud-Provider Container bereitstellen
 - Clouds bieten mehrere Varianten, Container auszuführen:
 - Container auf VMs (klassisch, volle Kontrolle)
 - Serverless Container-Dienste (z. B. AWS Fargate, Azure Container Apps, Cloud Run)
 - Container-Plattformen wie Kubernetes (EKS, AKS, GKE)
 - Gemeinsames Prinzip: die Cloud verwaltet Netzwerk, Skalierung und vieles vom Betrieb.
- Netzwerkgrundlagen bei Containern
 - Jeder Container bzw. jede „Dienstseinheit“ bekommt eine eigene IP oder Endpunkt-Adresse.
 - Kommunikation zwischen Containern im selben Dienst/Cluster erzeugt viel interne Kommunikation (East-West-Traffic).
 - Externe Zugriffe erfolgen über Load Balancer oder API-Gateways.
- Sicherheit im Container-Netz
 - Cloud-Netzwerkregeln (Security Groups / NSGs / Firewall Rules) schützen die Nodes oder Ausführungsumgebungen.
 - Feingranulare Regeln definieren, welche Container oder Dienste miteinander kommunizieren dürfen

Serverless / Functions-as-a-Service im Netz

- Grundidee von Serverless
 - Ausgeführt wird reiner Code (z. B. Python, Java, Node.js) als Funktion.
 - Keine Server, VMs oder Container werden selbst betrieben – diese Schicht übernimmt der Cloud-Provider.
- Wie Serverless in der Cloud aufgerufen wird
 - Funktionen werden typischerweise ausgelöst durch:
 - HTTP-Anfragen über ein API-Gateway
 - Ereignisse wie Messages, Timers, File-Uploads oder Queue-Events oder AI-Agents
 - Jede Ausführung läuft in einer isolierten, kurzlebigen Umgebung, die automatisch bereitgestellt wird.
- Netzwerkverhalten
- Inbound: meist über API-Gateway oder ähnliche Entry-Points; direkte IP-Erreichbarkeit existiert nicht.
- Outbound:
 - entweder direkt ins Internet
 - oder über NAT-Gateway / VPC- bzw. VNet-Integration, wenn interne Systeme erreicht werden sollen (z. B. Datenbank im privaten Subnet).
- Sicherheit
 - Zugriffskontrolle erfolgt primär über RBAC
 - Für interne Systeme wird Serverless häufig in private Cloud-Netze integriert (z. B. Lambda in VPC-Subnets), sodass normale Cloud-Firewall- / SSE-Regeln greifen.

Architektur von Public Clouds

- Cloud-Anbieter betreiben weltweit viele Rechenzentrums-Standorte.
- **Hierarchie:** Geography → Region → Availability Zones → Rechenzentren
- **Geography**
 - Fassen mehrere Regionen mit gleichen rechtlichen Rahmenbedingungen zusammen (z. B. Switzerland, Germany).
 - Dienen vor allem der Datenlokalität und Compliance, nicht der technischen Segmentierung
- **Regionen**
 - Geografische Gebiete (z. B. North Europe, Switzerland North).
 - Daten und Workloads verbleiben in der gewählten Region
 - Beinhalten Availability Zones für Redundanz die innerhalb der Region “gekoppelt” sind (Optional: Geo-Redundanz -> Verteilung auf mehrere Regionen)
- **Availability Zones (AZs)**
 - Mehrere, räumlich getrennte Rechenzentren innerhalb einer Region.
 - Eigene Stromversorgung/Netzwerkanbindung, aber niedrige Latenz untereinander.
 - Provider verteilen Workloads (automatisch) auf mehrere Zonen und replizieren diese automatisch (optional single Zone)
- **Rechenzentren**
 - In jeder AZ mehrere Rechenzentren mit Racks aus Servern, Storage, Netzwerk.
 - Auf diesen Servern laufen Hypervisor, Container-Plattformen und SDN-Fabric.



Begriffe in Public Clouds

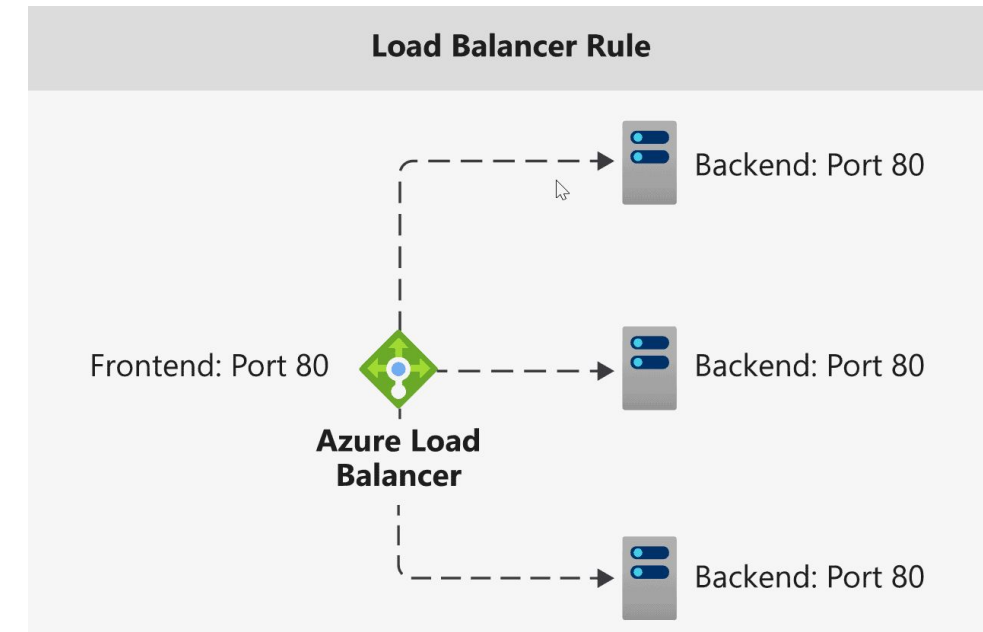
- Subscription / Project / Account-Ebene
 - Ebene für Abrechnung & Policies
 - Trennung von Prod/Test/Mandanten
- Resource Groups / Projekte (logische Container)
 - Bündeln Ressourcen für Struktur, Berechtigungen, Lifecycle
- IAM-Scope & Sicherheitsgrenzen
 - Rollen/Policies greifen auf verschiedenen Ebenen: Tenant → Subscription → Resource Group → Ressource.
 - Wichtig, um zu verstehen, wo man Netzwerk- und Sicherheitsrichtlinien verankert
- Netzwerk-Segmentierung & Konnektivität
 - Public vs. Private Subnets, Security Groups / NSG / Firewalls, optional NACLs
 - Externe Anbindung: Internet-/NAT-Gateway, VPN, dedizierte Leitungen.

Virtuelle Netzwerke: VNet & Subnet

- **VNet** (Virtual Network)
 - Logisch isolierter IP-Adressraum in einer Cloud-Region
 - Enthält Subnets, Routing-Tabellen, Security-Regeln, Gateways
 - Entspricht grob einem Netzwerk im Rechenzentrum (Layer-3-Segment)
- **Subnet** (Subnetz im VNet)
 - IP-Teilbereich (CIDR) innerhalb eines VNet
 - Meist einer AZ zugeordnet; Zuordnung von VMs, PaaS-Diensten etc.
 - Steuert, welche Routing-Tabellen und Security-Policies gelten
- **Typische Nutzung** von Subnets
 - Trennung nach Rolle: Web / App / DB / Management
 - Public Subnets: direkte/indirekte Erreichbarkeit aus dem Internet (z. B. via Load Balancer)
 - Private Subnets: nur interner Zugriff, Ausgehend-Traffic über NAT / Proxy
- **Designprinzipien**
 - Genug IP-Reserve einplanen (nicht zu kleine Netze)
 - Keine überlappenden IP-Bereiche zwischen VNets/On-Premises
 - „Least Privilege“: Kommunikation zwischen Subnets gezielt über Security-Gruppen/Firewalls zulassen

Load Balancer & VNet-„Stretching“ über Availability Zones

- **VNet-„Stretching“**
 - Ein VNet ist regional und umfasst alle Availability Zones einer Region.
 - Subnets sind ebenfalls regional; VMs in Zone 1 und Zone 2 können im gleichen VNet/Subnet liegen.
- **Load Balancer im VNet**
 - Stellen eine Frontend-IP (intern oder öffentlich) bereit und verteilen Traffic auf mehrere Instanzen im VNet.
 - Werden explizit erstellt und konfiguriert (Backend-Pool, Regeln, Health-Probes); Backends behalten eigene private IPs.
- **Zonenübergreifende Hochverfügbarkeit**
 - Backends laufen in mehreren Zonen derselben Region, aber im selben VNet.
 - Der regionale Load Balancer erreicht alle Zonen; bei Zonen-Ausfall sendet er nur noch zu gesunden Instanzen.
- **Merksatz**
 - Ein VNet spannt sich über alle Zonen einer Region; ein Load Balancer bündelt mehrere zonenverteilte Instanzen hinter einer einzigen Service-IP.



Standard Routing in Public Clouds

- **Routing Grundprinzip**

- Jedes Subnetz hat genau eine eigene Routentabelle (eine Tabelle kann an viele Subnetze gehängt werden)
- Die Cloud legt diese Routen logisch auf die NIC der Ressource (Bsp. VM).
- Ressourcen im gleichen Subnetz sprechen immer direkt miteinander (ähnlich Layer-2)
- Routing Ziele sind:
 - IPs oder VMs
 - VNets
 - Internet
 - Virtual Network Gateway
 - None

- **Typische Einsatzszenarien**

- Hub-Spoke mit zentraler Firewall (UDR zwingt Traffic durch Hub).
- Forced Tunneling von 0.0.0.0/0 über On-Premises oder Security-Appliance.
- Blackholing unerwünschter Netze mit Next Hop „None“.

Segmentierung in Public Clouds

- **Security Grundprinzip**

- Filterung auf mehreren Ebenen: NIC - Subnet, VNet - Perimeter
- Ressourcen werden Gruppen zugeordnet
 - Security Groups (n:n): Für VMs & Scale-Sets (mehrere VMs gleicher Art)
 - Tags (n:n): Ressourcen können getagged werden ("Allow_Internet_Access") und diese Regeln in Tags verwendet werden
- Policy-Gruppen:
 - Gruppieren einzelne Regeln und haben eine Priorität.
 - Die Priorität gibt die Ausführungsreihenfolge vor, womit sich globale Regeln (DROP all Internet 2 Admin-Interfaces) Regeln definieren lassen ohne den Überblick zu verlieren
- Zentrale Firewalls sind optional -> Hub & Spoke

- **Einsatzszenarien**

- Mikrosegmentierung im Subnet (NSG+ASG für Web/App/DB)
- Hub-Spoke mit zentraler Firewall/WAF für Internet- und Ost/West-Traffic
- „Default Deny“: nur explizit erlaubte Flows → Basis für Zero-Trust-Architekturen“.

Hub-and-Spoke-Prinzip in Clouds

- Motivation:
 - Zentrales Security-Hub statt komplexer voll-vermaschter Subnetze
 - Einfacheres Management, gerade in hybriden Umgebungen
- Topologie:
 - Hub-Netz:
 - enthält zentrale Komponenten: Firewalls, VPN-Terminierungen, Shared Services (AD, DNS, Logging)
 - Spoke-Netze:
 - enthalten Workloads für einzelne Applikationen
- Routing:
 - Spokes haben Standardroute (0.0.0.0/0) → Hub
 - Hub entscheidet, ob Traffic:
 - ins Internet
 - zu anderen Spokes
 - zu On-Prem geht
- Vorteile:
 - einheitliche Policy & Logging im Hub
 - klare Trennung der Workloads in Spokes

Hub-and-Spoke-Prinzip in Clouds

- Motivation:
 - Zentrales Security-Hub statt komplexer voll-vermaschter Subnetze
 - Einfacheres Management, gerade in hybriden Umgebungen
- Topologie:
 - Hub-Netz:
 - enthält zentrale Komponenten: Firewalls, VPN-Terminierungen, Shared Services (AD, DNS, Logging)
 - Spoke-Netze:
 - enthalten Workloads für einzelne Applikationen
- Routing:
 - Spokes haben Standardroute (0.0.0.0/0) → Hub
 - Hub entscheidet, ob Traffic:
 - ins Internet
 - zu anderen Spokes
 - zu On-Prem geht
- Vorteile:
 - einheitliche Policy & Logging im Hub
 - klare Trennung der Workloads in Spokes

Anbindung von Public Clouds

- **Internetbasierter Zugriff**
 - Zugriff über öffentliche Netze mit verschlüsselten Protokollen (HTTPS, SSH, VPN-Client), häufig ergänzt durch SSE/SASE bzw. ZTNA für Benutzerzugriffe.
- **Tunnelbasierter Zugriff (IPsec, GRE)**
 - Tunnel zwischen On-Prem-Firewall/Router/SD-WAN und Cloud-VPN-Gateway
 - Flexible, kostengünstige Kopplung über das Internet, dynamisches Routing meist via BGP.
- **Dedizierte Leitungen**
 - Provider-spezifische Privatverbindungen (z. B. ExpressRoute, Direct Connect, Interconnect) mit garantierter Bandbreite und geringerer Latenz als Internet-VPN.
- **SASE-Anbindung**
 - Für User-Zugriffe: SSE
 - Für East-West zwischen Clouds und Standorten WAN-EDGE (Alias SD-WAN).

Content Delivery Netzwerke (CDN)

- **Definition**

- Global verteiltes Netzwerk von Edge-Servern, das Inhalte (z. B. Bilder, Videos, JavaScript, CSS) möglichst nah beim Nutzer zwischenspeichert.

- **Funktionsprinzip**

- DNS/Anycast lenkt Anfragen zur geografisch nächsten Edge-Location (geringe Latenz)
- Bei Cache-Hit: Auslieferung direkt vom Edge, bei Cache-Miss: Weiterleitung zum Origin-Server und anschliessendes Caching.

- **Vorteile**

- Reduzierte Latenz und schnellere Ladezeiten, Entlastung des Origin-Servers, höherer Durchsatz bei Lastspitzen, oft zusätzlicher DDoS- und TLS-Schutz am Edge.

- **Einsatzszenarien**

- Web- und Mobile-Frontends, Medien-Streaming, Software-Downloads, globale APIs (mit selektivem oder teilweisem Caching).

Content Delivery Netzwerk

